

[Vai al contenuto](#)

DISPENSA DELLA LEZIONE

Relazioni tra tabelle con MySQL

Come collegare dati distribuiti in tabelle diverse con chiavi esterne e cardinalità.



CAPITOLO 1

Perché usare relazioni

NOTA

Una relazione collega record di tabelle diverse senza duplicare gli stessi dati in più punti.

In un database reale le informazioni appartengono spesso a entità diverse. Un cliente e un ordine, per esempio, sono due cose distinte: ciascuna merita una tabella con i propri campi.

Salvare i dati del cliente dentro ogni ordine produrrebbe duplicazioni. Se l'email cambia, sarebbe necessario correggerla in tutti gli ordini. Separando le tabelle, i dati del cliente vengono registrati una sola volta e gli ordini si collegano a quel record.

clienti

```
+---+-----+
| id | email      |
+---+-----+
| 1  | mario@email.it |
+---+-----+
```

ordini

```
+---+-----+-----+
| id | totale  | cliente_id |
+---+-----+-----+
| 1  | 49.90   | 1          |
+---+-----+-----+
```

CAPITOLO 2

Chiave primaria e chiave esterna

NOTA

La **chiave esterna** è un campo che contiene il valore della chiave primaria di un record presente in un'altra tabella.

La **chiave primaria** identifica in modo univoco un record nella propria tabella. In `clienti`, per esempio, `id` identifica un singolo cliente.

La **chiave esterna**, o *foreign key*, crea il collegamento. In `ordini`, il campo `cliente_id` contiene l'`id` del cliente a cui appartiene l'ordine.

```
clienti.id ← ordini.cliente_id
```

MySQL può controllare questo collegamento: un ordine non può riferirsi a un cliente che non esiste. Questa garanzia si chiama **integrità referenziale**.

CAPITOLO 3

Relazione uno a molti

NOTA

In una relazione uno-a-molti, un record della prima tabella può essere collegato a molti record della seconda; ogni record della seconda tabella appartiene a uno solo della prima.

Un cliente può effettuare molti ordini, ma ogni ordine appartiene a un solo cliente. La chiave esterna viene quindi inserita nella tabella che rappresenta la parte "molti": **ordini**.

```
CREATE TABLE clienti (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  nome VARCHAR(50) NOT NULL,  
  email VARCHAR(100) NOT NULL UNIQUE  
);  
  
CREATE TABLE ordini (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  totale DECIMAL(10, 2) NOT NULL CHECK (totale >= 0),  
  cliente_id INT NOT NULL,  
  FOREIGN KEY (cliente_id) REFERENCES clienti(id)  
);
```

Il tipo di **cliente_id** deve essere compatibile con il tipo della chiave primaria a cui si riferisce. Qui entrambi sono **INT**.

CAPITOLO 4

Relazione uno a uno

NOTA

In una relazione uno-a-uno, un record può essere collegato al massimo a un solo record dell'altra tabella.

Questo tipo di relazione è meno frequente. Può essere utile quando alcuni dati sono opzionali, riservati o usati raramente. Per esempio, un utente può avere un solo profilo esteso.

Per rendere univoco il collegamento, la chiave esterna deve avere anche il vincolo **UNIQUE**.

```
CREATE TABLE utenti (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  email VARCHAR(100) NOT NULL UNIQUE  
);  
  
CREATE TABLE profili (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  biografia TEXT,  
  utente_id INT NOT NULL UNIQUE,  
  FOREIGN KEY (utente_id) REFERENCES utenti(id)  
);
```

CAPITOLO 5

Modifiche e cancellazioni

NOTA

Le azioni della chiave esterna definiscono che cosa accade ai record collegati quando il record di riferimento viene modificato o eliminato.

Le azioni si dichiarano dopo **REFERENCES**. Devono rispecchiare la regola del dominio: non esiste una scelta sempre corretta.

```
FOREIGN KEY (cliente_id)
  REFERENCES clienti(id)
  ON DELETE RESTRICT
  ON UPDATE CASCADE
```

- **RESTRICT** blocca l'operazione se esistono record collegati.
- **CASCADE** propaga l'operazione ai record collegati.
- **SET NULL** imposta a **NULL** la chiave esterna; il campo non può quindi avere **NOT NULL**.

ON DELETE CASCADE può eliminare molti record insieme: va usato soltanto quando quei record non hanno significato senza il record di riferimento.

CAPITOLO 6

Relazione molti a molti

NOTA

Una relazione molti-a-molti richiede una terza tabella, chiamata **tabella ponte**, che contiene le due chiavi esterne.

Uno studente può frequentare molti corsi e un corso può avere molti studenti. Non è corretto salvare una lista di corsi in un solo campo dello studente: ogni campo deve contenere un solo valore.

studenti ← iscrizioni → corsi

```
CREATE TABLE studenti (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  nome VARCHAR(50) NOT NULL  
);  
  
CREATE TABLE corsi (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  nome VARCHAR(100) NOT NULL  
);  
  
CREATE TABLE iscrizioni (  
  studente_id INT NOT NULL,  
  corso_id INT NOT NULL,  
  data_iscrizione DATE NOT NULL,  
  PRIMARY KEY (studente_id, corso_id),  
  FOREIGN KEY (studente_id) REFERENCES studenti(id),  
  FOREIGN KEY (corso_id) REFERENCES corsi(id)  
);
```

La chiave primaria composta da **studente_id** e **corso_id** impedisce di registrare due volte la stessa iscrizione. La tabella ponte può inoltre avere attributi propri, come **data_iscrizione**.